南開大學
Nankai University

计算机学院

并行程序设计实验报告

体系结构相关实验分析

姓名：黄婉文

学号：2413406

专业：计算机科学与技术

2026 年 3 月 28 日

目录

1 实验平台 **2**

2 实验一：n*n 矩阵与向量内积 **2**

2.1 算法设计 2

2.1.1 平凡算法 2

2.1.2 cache 优化算法 3

2.2 编程实现 3

2.2.1 平凡算法 3

2.2.2 cache 优化算法 4

2.3 性能测试 4

2.3.1 平凡算法 4

2.3.2 cache 优化算法 5

2.4 Profiling 5

2.5 结果分析 6

2.5.1 平凡算法 6

2.5.2 cache 优化算法 7

2.6 算法对比与汇编分析 7

3 实验二：n 个数求和 **7**

3.1 算法设计 7

3.1.1 平凡算法 7

3.1.2 超标量优化算法 8

3.2 编程实现 8

3.2.1 平凡算法 8

3.2.2 超标量算法 8

3.3 性能测试 9

3.3.1 平凡算法 9

3.3.2 超标量算法 9

3.4 Profiling 10

3.5 结果分析 10

3.5.1 平凡算法 10

3.5.2 超标量算法 10

3.6 算法对比与汇编分析 11

4 实验总结和思考 **12**

4.1 异同对比 12

4.2 总结 12

5 附：大模型对话链接 **12**

摘要

本实验通过矩阵向量内积与 n 个数求和两个问题，探究访存模式与指令级并行对程序性能的影响。实验采用高精度计时与 Intel VTune 性能剖析，对比了不同算法的执行时间与微架构指标。结果表明，矩阵内积中 cache 优化算法通过改善空间局部性，大大降低了 cache miss 次数；n 个数求和证明了超标量算法的并行有效性，其中四路链式算法最优。实验验证了算法性能不仅取决于时间复杂度，更取决于与硬件特性的匹配程度，为并行程序设计提供了量化依据。

github 链接： https://github.com/fengxin9/parallel-program-lab/tree/main/lab1

1 实验平台

实验环境：

- **主平台（x86）**：Windows 11 + Visual Studio Code + MinGW-w64 (GCC 8.1.0)
- **对比分析 1（x86 Linux）**：Ubuntu 24.04 LTS (VMware 虚拟机) + GCC 13.3.0
- **对比分析 2（ARM 模拟）**：Windows 11 + WSL 2 + QEMU，交叉编译工具链 aarch64-linux-gnu-gcc 13.3.0，模拟 ARM64 平台

CPU 信息：

- **型号**：12th Gen Intel(R) Core(TM) i5-1235U(2.50 GHz)，10 核 12 线程
- **内存**：16.0GB
- **CPU 缓存配置**：L1 缓存：80KB/核，L2 ： 6.5 MB，L3 ： 12 MB

2 实验一：n*n 矩阵与向量内积

2.1 算法设计

2.1.1 平凡算法

本实验实现了矩阵与向量内积的平凡算法。给定一个 n×n 的矩阵 mat 和一个 n 维向量 a，计算算每一列与向量的内积得到结果向量 $sum_j = \sum_{i=0}^{n-1} mat_{i,j} \cdot a_i$。算法采用列优先访问方式，外层循环遍历列，内层循环遍历行，每次累加当前列元素与对应向量分量的乘积。

Algorithm 1 平凡算法：矩阵与向量内积

Input: 矩阵 $mat[n][n]$，其中 $mat[i][j] = i + j$；向量 $a[n]$，其中 $a[i] = i$

Output: 结果数组 $sum[n]$

```
function MATRIX_PRODUCT(mat, a, sum, n, outfile)
    for j ← 0 to n − 1 do
        sum[j] ← 0
        for i ← 0 to n − 1 do
            sum[j] ← sum[j] + mat[i][j] × a[i]
        end for
    end for
```

```
    return sum
end function
for k ← 1 to m do
    初始化矩阵和结果数组
    调用计算函数并使用 QueryPerformanceCounter() 计时
    将 sum 写入文件（防止编译器优化）
    time_sum ← time_sum + time
end for
return avg_time = time_sum/m
```

2.1.2 cache 优化算法

Cache 优化算法采用行优先访问方式计算矩阵与向量的内积。与平凡算法不同，该算法外层循环遍历行，内层循环遍历列，每次将当前行的贡献累加到所有列的结果中。

Algorithm 2 cache 算法：矩阵与向量内积

Input: 矩阵 $mat[n][n]$，其中 $mat[i][j] = i + j$；向量 $a[n]$，其中 $a[i] = i$

Output: 结果数组 $sum[n]$

```
function MATRIX_PRODUCT(mat, a, sum, n, outfile)
    for j ← 0 to n − 1 do
        sum[j] ← 0
    end for
    for i ← 0 to n − 1 do
        for j ← 0 to n − 1 do
            sum[j] ← sum[j] + mat[i][j] × a[i]
        end for
    end for
    return sum
end function
```

2.2 编程实现

2.2.1 平凡算法

逐列访问平凡算法

```
double matrix_product(int **mat, int *a, long long *sum, int n, ofstream& outfile) {
  long long head, tail, freq;
  QueryPerformanceFrequency((LARGE_INTEGER *)&freq);   // 获取计时器频率
  QueryPerformanceCounter((LARGE_INTEGER *)&head);     // 计时器初始值
  for(int i=0;i<n;i++){
      sum[i] = 0;
      for(int j=0;j<n;j++){
          sum[i] += mat[j][i] * a[j];
      }
  }
  for(int i = 0; i < n; i++) {  // 将结果写入文件，防止编译器优化
```

```
        outfile << sum[i] << " ";
        if(i % 10 == 9) {
            outfile << endl;
        }
    }
    QueryPerformanceCounter((LARGE_INTEGER *)&tail);    // 计时器结束值
    double time = (double)(tail - head) / freq;  // 计算运行时间
    return time;
}
```

2.2.2 cache 优化算法

逐行访问 cache 算法

```
for(int i=0;i<n;i++)
        sum[i] = 0;
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            sum[j] += mat[i][j] * a[i];
        }
    }
// 其余输入输出、文件写入和计时均与平凡算法相同
```

2.3 性能测试

由于单次计算用时较短，因此用重复一百次的时间总和作为衡量标准。

1. 编译选项（优化力度）：为探究编译器优化级别对程序性能的影响，分别使用 -O0（无优化）、-O1（基础优化）、-O2（推荐优化）和 -O3（激进优化）四种编译选项对同一算法进行测试，对比不同优化力度下的执行时间差异。O0 编译速度快，便于调试，不进行任何优化；O1 减少代码大小和执行时间，不进行会显著增加编译时间的优化；O2 执行所有不包含空间-时间权衡的优化；O3 在 O2 基础上增加了循环展开、函数内联等优化。

2. 不同平台：在 x86 Linux 虚拟机平台和 Linux 模拟 64 位 ARM 平台上进行 SIMD 和多线程程序编译，优化力度为 O2，矩阵大小为 1000*1000，分别用 SSE/AVX（ARM 中为 Neon）、Pthread、OpenMP 的编译选项进行测试（具体编译命令见代码文件）。由于 x86_64 系统需要用 QEMU 模拟运行，必须使用 static 静态编译命令。

2.3.1 平凡算法

1. 不同优化力度：

如表 1所示，总体性能表现 O3>O2>O1>O，无优化性能明显弱于有优化，矩阵大小在 10^3 以上时 O0、O1 优化和其他差距明显，O2、O3 差距不大。另外可以观察到算法运行时间随规模平方级增长，即 $O(n^2)$。

2. 不同平台：

编译选项 \ 矩阵大小 n	10	100	1000	5000	10000
-O0	0.0027987	0.011957	0.454597	26.5699	190.385
-O1	0.0016169	0.0065169	0.347602	18.1085	116.723
-O2	0.0010562	0.0048294	0.212762	14.8579	75.3924
-O3	0.0006296	0.0046954	0.184703	14.8554	75.37

表 1: 性能测试结果 (平凡算法)(单位:s)

如表 2所示，Linux 平台执行速度比 ARM 模拟平台快约 30 倍，其中 Linux 平台 Pthread 版本最优。性能差异主要源于 Linux 运行于真实硬件，而 ARM 通过 QEMU 模拟运行，模拟器开销较大，仅适用于正确性验证，不宜用于性能评测。

平台 \ 编译选项	SSE/AVX(Neon)	Pthread	OpenMP
Linux	0.13923	0.123847	0.161084
ARM	4.53743	3.92191	3.58582

表 2: 平台对比 (平凡算法)(单位:s)

2.3.2 cache 优化算法

1. 不同优化力度：

如表 3所示，性能表现仍然是 O3>O2>O1>O，但是整体显著优于平凡算法，相同优化级别下性能提升明显，尤其是 10^3 以上规模时运算时间只有平凡算法约十分之一，验证了访存优化对并行程序性能的重要性。

编译选项 \ 矩阵大小 n	10	100	1000	5000	10000
-O0	0.00084	0.0116232	0.22993	4.51504	17.4124
-O1	0.001426	0.0069713	0.104104	1.75282	6.55993
-O2	0.0013929	0.0085558	0.103263	1.71709	6.43022
-O3	0.000577	0.0072459	0.0814771	1.30796	4.51855

表 3: 性能测试结果 (cache 算法)(单位:s)

2. 不同平台：

如表 4所示，cache 优化算法在 Linux 和 ARM 平台上的执行时间显著优于平凡算法，加速比约为 2 倍，验证了行优先访问策略对 cache 性能的提升效果。但提升幅度相对 x86 平台较小，进一步说明模拟器环境下访存优化的效果受到限制。

2.4 Profiling

使用 Vtune 工具的 Hotspots（热点分析）和 Memory access（内存访问分析）功能对两种算法进行性能剖析，均采用 O2 优化力度，矩阵规模 1000，重复 100 次。主要分析参数有：程序运行时间

平台 \ 编译选项	SSE/AVX(Neon)	Pthread	OpenMP
Linux	0.0671701	0.0643103	0.0701976
ARM	4.09891	3.2641	3.33488

表 4: 平台对比 (cache 算法)(单位:s)

Elapsed Time，CPU 时间 CPU Time，CPU 实际执行的指令数量 Instructions Retired，CPU 微架构利用率 Microarchitecture Usage，内存访问导致的停顿比例 Memory Bound，cache 未命中导致的停顿比例 L1/L2/L3 Bound，主存访问导致的停顿比例 DRAM Bound，Store Bound，L3 未命中次数 LLC Miss Count 等。

如表 5所示，平凡算法的整体性能较差，程序运行时间长，cache miss 次数很高，高达三百多万次；而 cache 算法的性能远优于平凡算法，程序执行时间更短，cache miss 次数为 0，命中率很高。

表 5: 平凡算法与 Cache 优化算法 VTune 对比

指标	**平凡算法**	**Cache 优化算法**	**变化**
Elapsed Time (s)	3.745	3.068	↓ 18.1%
CPU Time (s)	0.347	0.275	↓ 20.7%
Instructions Retired	2.276B	2.157B	↓ 5.3%
Microarchitecture Usage	27.1%	35.1%	↑ 29.5%
Memory Bound	41.1%	24.6%	↓ 40.1%
DRAM Bound	26.9%	13.2%	↓ 50.9%
L1 Bound	5.5%	11.2%	↑ 103.6%
L2 Bound	3.2%	2.0%	↓ 37.5%
L3 Bound	2.4%	4.1%	↑ 70.8%
Store Bound	2.4%	3.0%	↑ 25.0%
Loads	470M	384M	↓ 18.3%
Stores	221M	327M	↑ 48.0%
LLC Miss Count	3.85M	0	↓ 100%

2.5 结果分析

2.5.1 平凡算法

根据 profiling 的结果，平凡算法的程序执行时间为 3.745 秒，而 CPU 时间仅为 0.347 秒，二者相差约十倍，表明大量时间消耗在内存访问等待上，CPU 资源利用率较低；Memory Bound 高达 41.1%，其中 DRAM Bound 占 26.9%，说明主内存访问是主要性能瓶颈；极高的 LLC Miss Count 进一步证实了频繁的末级缓存未命中。各级缓存未命中率较低是由于列优先访问模式导致数据无法有效利用缓存，大量数据直接从主内存读取，绕过了缓存层次。

这是因为平凡算法采用列优先访问，由于 C/C++ 行优先存储的特性，每次访问矩阵都跨越整行数据，空间局部性极差，无法有效利用 CPU 缓存，造成频繁的主内存访问，访存模式与硬件存储方式

不匹配，导致缓存失效和主内存访问延迟，严重制约了程序性能。

2.5.2 cache 优化算法

Cache 优化算法的性能远优于平凡算法，程序执行时间相比平凡算法减少了 18.1%，CPU 时间减少 20.7%，CPU 资源利用率大大提高；Memory Bound 降幅达 40.1%，其中 DRAM Bound 减半，表明主内存访问压力显著降低；L1 Bound 升高，说明行优先访问使数据更多驻留在 L1 缓存中；LLC Miss Count 降至 0 说明所有数据均命中最后一级缓存，无需访问主内存，体现了算法优化的效果。

cache 优化算法通过行优先访问策略，显著改善了空间局部性，与 C/C++ 行优先存储的特点匹配，使数据能够充分利用 CPU 缓存层次，将 cache miss 降至零，主内存访问减半，实现了极大的性能提升。

2.6 算法对比与汇编分析

图 2.1比较了在 O0、O1、O2 优化力度下平凡算法和 cache 算法复杂度随问题规模增长的趋势。

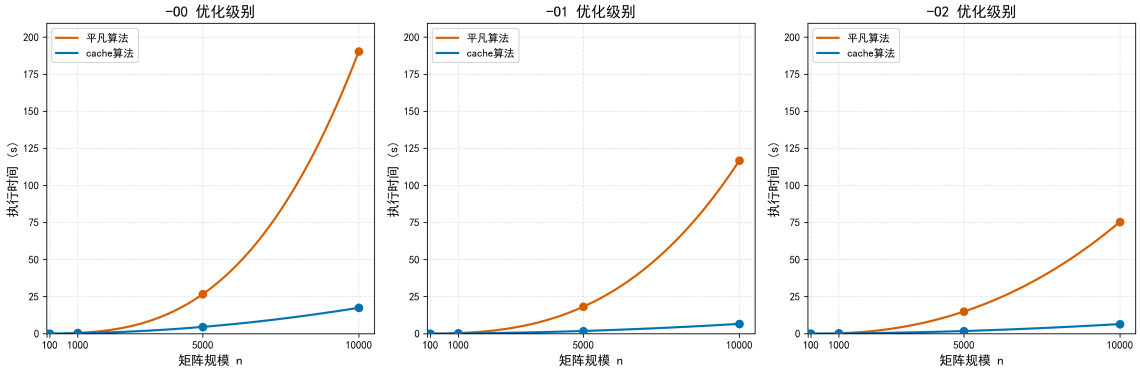

图 2.1: 不同优化力度下算法运行时间的对比

可见，在进行算法设计时除了要考虑算法本身的复杂度，还要考虑算法与硬件的匹配程度。这解释了为什么平凡算法与 cache 优化算法虽然具有相同的时间复杂度 $O(n^2)$，cache 算法却表现更优。而在 linux 平台下两种算法的性能相较在 x86 平台略有提升，但是 ARM 模拟平台下明显更慢，说明程序执行速度受到了硬件模拟的限制。

另外通过 https://godbolt.org 网站生成对应函数的汇编程序中显示，两种算法的核心计算指令基本相同，但访存模式不同。平凡算法因跨行访问导致每次加载都可能触发缓存缺失，而 cache 优化算法的连续访问使数据能高效驻留在 L1/L2 缓存中。对应汇编程序已上传 github。

3 实验二：n 个数求和

3.1 算法设计

3.1.1 平凡算法

本实验需要计算 n 个数的和，首先初始化需要计算的数组，平凡算法即链式算法采用逐个累加的方式，循环 n 次。为方便递归算法，n 取 2 的幂次。

Algorithm 3 平凡算法：n 个数求和

Input: 数组 $a[n]$，其中 $a[i] = i$
Output: 结果 sum

function MATRIX_PRODUCT($a, sum, n, outfile$)
  $sum \leftarrow 0$
  for $i \leftarrow 0$ to $n - 1$ **do**
    $sum \leftarrow sum + a[i]$
  end for
  return sum
end function
其余部分与实验一相同

3.1.2 超标量优化算法

优化算法采用适合超标量架构的指令级并行算法，相邻指令无依赖。具体实现方式为多路链式累加和递归算法：多路链式相加指将待加数组分为多路，一次循环内分别计算几路数组的和，最后将结果相加；递归算法指两两相加、中间结果再两两相加，依次类推，直至只剩下最终结果。递归算法有使用递归函数和二重循环两种实现方式，前者相对简单，后者开销较小。具体实现见 3.2。

3.2 编程实现

3.2.1 平凡算法

累加平凡算法

```
double ordinary_sum(int *a, long long &sum, int n, ofstream& outfile) {
  sum = 0;
  for (int i = 0; i < n; i++) {
      sum += a[i];
  }
  outfile << sum << endl;
}
```

3.2.2 超标量算法

1. 多路链式相加——以两路链式相加为例：

多路链式相加算法

```
double two_chain(int *a, long long &sum, int n, ofstream& outfile) {
  long long sum1 = 0, sum2 = 0;
  for(int i=0; i<n; i+=2){  // 由于数组大小为2的幂次，所以无需考虑奇数情况
      sum1 += a[i];
      sum2 += a[i+1];
  }
  sum = sum1 + sum2;
  outfile << sum << endl;
  /* 四路、八路链式相加只需将sum1、sum2改为对应大小的sum数组，令sum[j] += a[i+j]，
   最后将sum的所有元素累加即可 */
```

```
}
```

2. 递归算法：

递归算法

```
long long recursive_sum(int *a, int n) {    // 使用递归函数
    if (n == 1) {
        return a[0];
    }
    for(int i=0; i<n/2; i++){
        a[i] += a[n-i-1];    // 相邻元素相加连续存储到数组最前面
    }
    n/=2;
    return recursive_sum(a, n);
}

long long double_iter(int *a, int n) {    // 使用二重循环
    for(int m=n; m>1; m/=2){    // 共需要log2(n)轮
        for(int i=0; i<m/2; i++){
            a[i] = a[i*2] + a[i*2+1];
        }
    }
    return a[0];  // 相邻元素相加连续存储到数组最前面，最后结果存储在a[0]中
}
```

3.3 性能测试

本实验中 n 取 2 的 8、10、13、15、17 次幂进行测试，记录重复一万次的用时总和为结果。

3.3.1 平凡算法

如表 6所示，不同优化力度下性能有差异，但不如实验一明显，因为本实验的时间复杂度为 O(n)，程序执行时间随问题规模线性增长。而规模较大时 O1 优化力度下程序性能还不如 O0，可能是编译器对代码进行了无效优化。因此在超标量算法中不再测试编译选项，仅针对算法本身进行测试。

编译选项 \ 数组大小 n	128	512	4096	16384	65536
-O0	0.0399539	0.0431575	0.0729513	0.177894	0.559454
-O1	0.0371888	0.0449739	0.108914	0.329201	1.20595
-O2	0.0370984	0.0387979	0.0479869	0.0786003	0.206921
-O3	0.0355791	0.0369194	0.045699	0.0622488	0.119762

表 6: 性能测试结果 (平凡算法)(单位:s)

3.3.2 超标量算法

在 O2 优化力度下对不同算法、不同规模进行测试。

1. 多路链式累加：分别测试两路、四路、八路累加的效果。

如表 7所示，两路链式相加在小规模下和平凡算法差距不大，但规模增大时表现更优；四路的效果最好，相较平凡算法和两路链式算法都有提升；而八路链式算法虽然链数更大，性能表现却更差，说明分链操作本身增加了算法的复杂度，在给定规模下并没有体现出更大的优势。可见链数并不是越多越好，而是要寻找分链降低的复杂度与其本身引入的复杂度之间的平衡。

链数 \ 数组大小 n	128	512	4096	16384	65536
2	0.0378965	0.0396091	0.0429963	0.0657294	0.153154
4	0.036352	0.0383549	0.0468592	0.0635532	0.138291
8	0.0381405	0.0417047	0.0650609	0.152947	0.505211

表 7: 性能测试结果 (多路链式相加)(单位:s)

2. 递归：分别测试递归函数和二重循环两种实现方式的性能效果。

规模较小时，递归算法的性能明显优于平凡算法和多路链式累加算法，但规模较大 (n=65536) 时递归算法的表现不如多路链式算法，说明问题规模较大时递归算法的调用开销较高，高于其所带来的性能优化效果。而对于具体实现方式，递归函数和二重循环的差异并不明显。

方法 \ 数组大小 n	128	512	4096	16384	65536
递归函数	0.0009211	0.0021559	0.0175069	0.0578634	0.22201
二重循环	0.001643	0.003864	0.0151643	0.0565114	0.212409

表 8: 性能测试结果 (递归)(单位:s)

3.4 Profiling

使用 O2 优化力度，在 n=65536 的情况下用 vtune 进行 profiling。对于超标量优化算法：

1. 多路链式累加算法：根据性能测试结果，这里使用表现最优的四路链式累加算法进行测试。
2. 递归算法：由于递归函数和二重循环两种方式差距不大，方便起见采用递归函数进行测试。

3.5 结果分析

3.5.1 平凡算法

平凡算法执行时间为 3.597 秒，其 VTune 数据显示 Bad Speculation（错误预测）高达 24.7%，说明分支预测错误是主要性能瓶颈，每次循环的分支判断虽然简单，但在迭代中累积了大量预测失误，导致推测执行频繁清空，浪费了近四分之一的 CPU 周期。

3.5.2 超标量算法

多路链式累加算法运用了循环展开的思想，通过将循环体重复多次来减少循环控制指令的执行次数，从而提高程序性能。其中四路链式算法表现最优，执行时间仅 3.319 秒，其指令数减少至 53.7 亿条，错误预测降至 14.6%，证明循环展开有效减少了循环控制指令和分支预测错误，但其 Back-End Bound 升至 25.0%，表明后端执行单元是其性能瓶颈。

表 9: VTune 性能对比

指标	平凡算法	四路链式	递归算法
Elapsed Time (s)	3.597	3.319	3.626
Clockticks	1.378B	1.076B	1.430B
Instructions Retired	6.871B	5.371B	7.278B
CPI Rate	0.201	0.200	0.197
Retiring	50.0%	53.3%	100.0%
Bad Speculation	24.7%	14.6%	0.0%
Front-End Bound	4.2%	7.1%	15.0%
Back-End Bound	21.1%	25.0%	3.6%
Memory Bound	7.9%	0	0
Core Bound	13.2%	36.1%	100%

递归算法执行时间最长，达到了 3.626 秒，与在性能测试中的结果相符。尽管其 CPI 最优、Bad Speculation 为 0、Retiring 高达 100%，但指令数多达 72.8 亿条，其中大量指令为函数调用、栈帧操作等额外开销，导致 CPU 虽然满负荷运转，执行的却是大量无效工作。综上，四路链式算法通过减少指令数、改善分支预测，在三种算法中取得最优性能，验证了循环展开和指令级并行的有效性。

3.6 算法对比与汇编分析

图 3.2比较了在 O2 优化力度下平凡、四路链式、递归三种算法执行时间随问题规模增长的趋势。

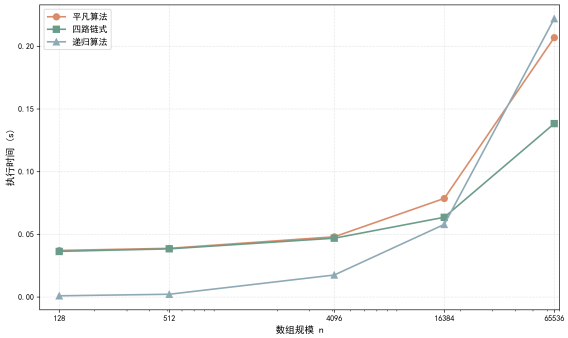

图 3.2: 不同算法运行时间的对比

从汇编程序上分析，两路链式算法的汇编代码将循环展开为每次处理两个元素，使用两个独立的累加器，内层循环中两条加法指令无数据依赖，可并行执行，同时循环次数减半，有效降低了循环控制指令的开销；递归算法的汇编代码最为复杂，包含函数调用、栈帧操作、参数传递和条件分支，当问题规模增大时，递归深度增加导致函数调用次数激增，大量占用了 CPU 执行资源。

从超标量的角度分析，本实验对比验证了超标量架构下指令级并行优化的有效性：平凡算法的性能瓶颈在于分支预测错误，每次循环的分支判断虽然简单，但在数十亿次迭代中累积了大量预测失误；而超标量算法通过减少循环开销、提高计算密度，使 CPU 资源从浪费转向充分利用，是提升程序性能的关键策略。

4 实验总结和思考

4.1 异同对比

两个实验虽然都是基础算术运算，但因计算特征和访存模式的不同，性能优化的侧重点也截然不同。矩阵向量内积实验的时间复杂度为平方级，计算密度高，涉及二维数组访问，性能瓶颈主要在于访存模式：平凡算法采用列优先访问，与 C/C++ 的行优先存储不匹配，导致空间局部性差、缓存命中率低。

n 个数求和实验的时间复杂度为线性级，计算密度低，是一维数组的顺序访问，访存压力小，性能瓶颈主要在指令级并行。平凡算法每次迭代仅处理一个元素，循环控制指令占比高；多路链式算法通过循环展开，减少了循环次数和循环时间；而递归算法虽在小规模时因树形结构优势表现最佳，但大规模下函数调用开销大，性能不佳。

4.2 总结

本实验通过矩阵向量内积与 n 个数求和两个问题，探究了现代计算机体系结构中算法性能与硬件特性的深层关联。实验结果表明，算法的时间复杂度并非决定性能的唯一因素，访存模式与硬件匹配程度、指令级并行能力同样关键：矩阵内积实验验证了空间局部性对 Cache 效率的决定性影响，n 个数求和实验证明了减少循环开销、提高计算密度是发挥超标量架构潜力的有效手段。

综上，本实验说明了算法设计不仅要考虑算法本身，还要考虑硬件情况，将访存模式、缓存层次、指令级并行等硬件特性纳入核心考量，实现算法与硬件的协同优化。

5 附：大模型对话链接

大模型：Deepseek-V3.2

对话链接：https://chat.deepseek.com/share/3c250af6tz7n93hokv